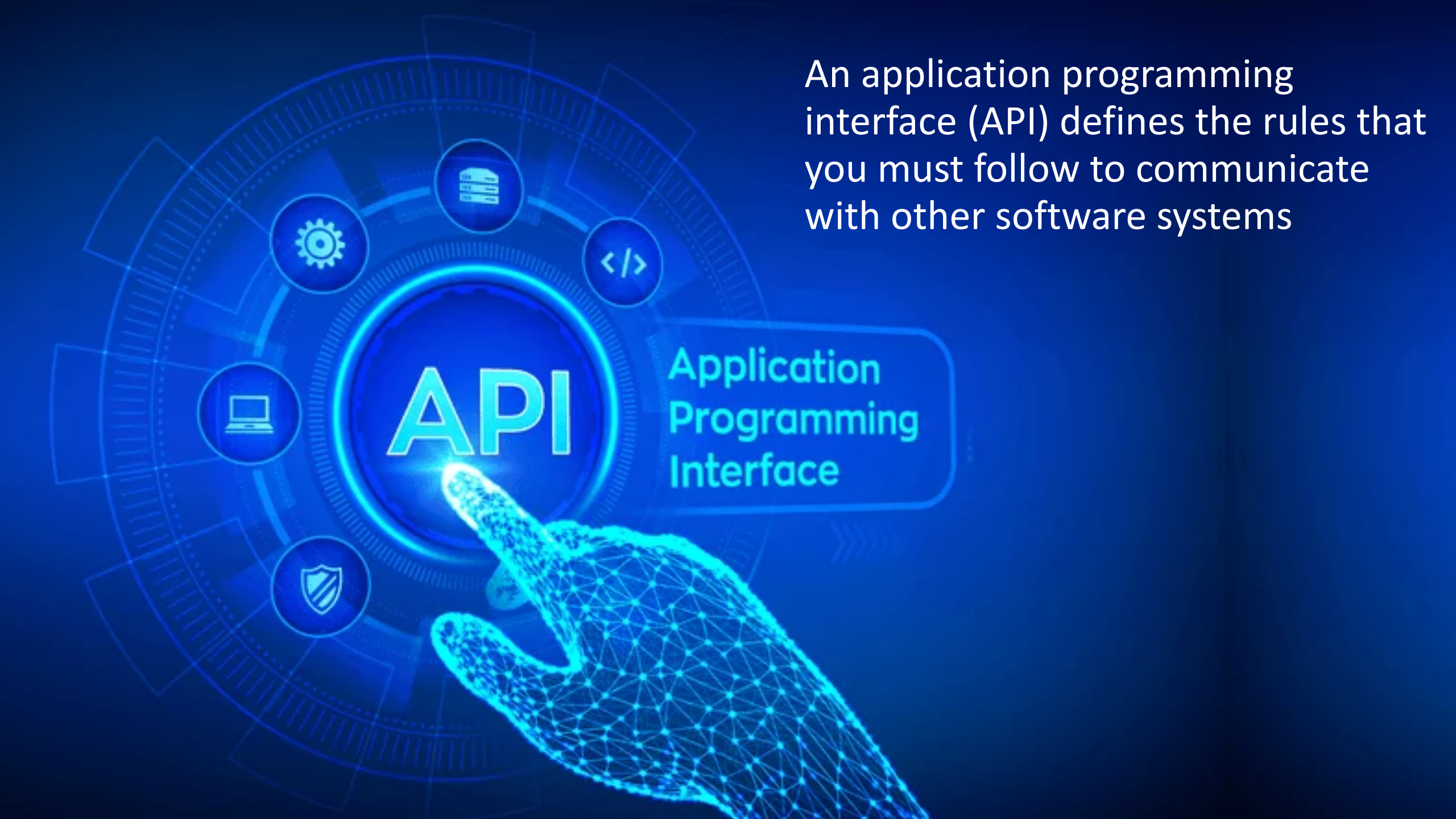
Principals in API Design

Ehsan Khamespanah

Spring 2026



An application programming interface (API) defines the rules that you must follow to communicate with other software systems

What is an API?

- Web API is a gateway between clients and resources on the web
- Clients: users who want to access information from the web
  - Can be a person or a software system that uses the API
- Resources: the information that different applications provide to their clients
  - Images, videos, text, numbers, or any type of data. The machine that gives the resource to the client is also called the server.
  - Organizations use APIs to share resources

What Is REST?

- Representational State Transfer (REST) is a software architecture that imposes conditions on how an API should work.
- RESTful APIs require requests to contain the following main components:
- Unique resource identifier: resource typically identification by using a URL
- Method: RESTful APIs usually developed using HTTP.
  The HTTP method tells the server what it needs to do to the resource.
  - GET: to access resources that are located at the specified URL on the server. They can cache GET requests

What Is REST?

- POST: sending data to the server
- PUT: updating existing resources on the server.
- DELETE: removing the resource.

- HTTP headers: the metadata exchanged between the client and server.
  - Data: requests might include data for the POST, PUT, and other HTTP methods to work successfully.
  - Parameters: Path parameters that specify URL details, Query parameters that request more information about the resource, Cookie parameters that authenticate clients quickly.

For the Flight Reservation System

Request	/flights	/flights/IR726
GET	List of flights	The details information of the flight IR726
POST	Create a new flight	Error
PUT	Bulk update of flights	If exists, update and if does not exist, error
DELETE	Delete all flights	Delete the flight IR726

Some Architectural Constraints

- Stateless: no data should be stored on the server during the processing of the request transfer. Meaning no client information is stored between requests and each request is separate and unconnected.
- Cacheable: The client should have the ability to store responses in a cache. This greatly improves the performance of the API.
- Uniform Interface: The uniform interface constraint is fundamental to the design of any RESTful system. It simplifies and decouples the architecture, which enables each part to evolve independently.

Constraints for Uniform Interface

- Resource identification in requests
  - Individual resources are identified in requests using URIs
  - The resources themselves are conceptually separate from the representations that are returned to the client
    Eg., the server could send data as JSON which is not its stored format
- Resource manipulation through representations
  - A client holds a representation of a resource, which is enough information to modify or delete the resource's state

Constraints for Uniform Interface

- Self-descriptive messages
  - Each message includes enough information to describe how to process the message.
- Hypermedia as the engine of application state (HATEOAS)
  - Having accessed an initial URI for the REST application, a client should then be able to use server-provided links dynamically to discover all the available resources
  - There is no need for the client to be hard-coded with information regarding the structure of the server

How to Design Services

- Having no standard works when there are 2-3 services
  - …/getAvailableSeats/Beijing
  - …/bookAFlight/Ali/Alavi/Beijing/2020-04-05/3
- But, we are going to expose all of the business logics using services
- Multiple developers design different services
- Providers and users are different (back-end and front-end developers are different)
- Before writing any Java code, it's crucial to internalise the constraints that define a services

What is a Resource?

- The key abstraction of information in REST is a resource.
- Any information that can be named, can be a resource:
  - A document or image, a temporal service, …
  - A collection of other resources, e.g., “today’s top selling products” which is collection of product resources
  - A non-virtual object, e.g., a person
- A resource can be a singleton or a collection.

```
/customers          //is a collection resource

/customers/{id}     //is a singleton resource
```

What is a Resource?

- A resource may contain sub-collection resources also

```
/customers      //is a collection resource

/customers/{id}/accounts  //is a sub-collection resource (customer's account)

/customers/{customerId}/accounts/{accountId} //a singleton resource
(customer's specific account)
```

Best Practices : Do’s and Don’ts

1. Use Proper HTTP Methods (Verbs) defining API operations
    - The differences between POST, PUT, and PATCH can be confusing.
2. Use nouns (plural nouns) to represent resources
    - URIs should refer to a resource that is a thing (noun) instead of referring to an action (verb): Users, Accounts
    - Concrete names are better than abstract names: “items”, “assets”, … are bad names

Best Practices : Do's and Don'ts

3. Focus on Readablity and Consistency
   - Do not use trailing forward slash (/) in URIs

```
GET /device-management/managed-devices/
GET /device-management/managed-devices    /*Much better version*/
```

   - Use hyphens (-) and Kebab-case (some-var-name) to improve the readability of URIs

```
GET   /devicemanagement/manageddevices/
GET   /device-management/managed-devices  /*Much better version*/

GET   /my-folder/my-doc     // Good
GET   /MY-FOLDER/my-doc     // Avoid
GET   /My-Folder/my-doc     // Avoid
```

Best Practices : Do’s and Don’ts

- Avoid underscore (_) and Camel-case (someVarName) Snake-case (some_var_name) Pasal-case (SomeVarName)

```
GET /managed-entities/{id}/install-script-location
                          //More readable
GET /managed_entities/{id}/install_Script_Location
                          //Snake-case - readable but not conventional
GET /managedEntities/{id}/installScriptLocation
                          //Camel-case - Less readable
```

- Do not use file extensions

```
/device-management/managed-devices.xml  /*Do not use it*/
/device-management/managed-devices      /*This is correct URI*/
```

Best Practices : Do's and Don'ts

- Never use CRUD function names in URIs

```
/*     Avoid  */
HTTP GET     /get-managed-devices
HTTP POST    /create-managed-devices


/*     Good   */
HTTP GET     /managed-devices          //Get all devices
HTTP POST    /managed-devices          //Create new Device

HTTP GET     /managed-devices/{id}     //Get device for given Id
HTTP PUT     /managed-devices/{id}     //Update device for given Id
HTTP DELETE  /managed-devices/{id}     //Delete device for given Id
```

Best Practices : Do’s and Don’ts

- Do not Use Verbs in the URI

```
GET    /scripts/{id}/execute //DON't DO THIS! – It is RPC, and not REST

POST   /scripts              //Better: Create a script using POST action

GET    /scripts/{id}/status  //check the execution status
```

4. Use HTTP Response Status Codes
   - 200 OK - A successful GET, PUT, PATCH, DELETE, and a POST that doesn't result in a creation.
   - 201 Created - Response to a POST that results in a creation

Best Practices : Do’s and Don’ts

- 204 No Content - Response to a successful request that won't be returning a body (like a DELETE request)
- 304 Not Modified - Used when HTTP caching headers are in play
- 400 Bad Request - The request is malformed, e.g., the body does not parse
- 401 Unauthorized - When no or invalid authentication details are provided
- 403 Forbidden - When authentication succeeded but authenticated user doesn't have access to the resource
- 404 Not Found - When a non-existent resource is requested

Best Practices : Do’s and Don’ts

- 405 Method Not Allowed - When an HTTP method is being requested that isn't allowed for the authenticated user
- 410 Gone - Indicates that the resource at this end point is no longer available. Useful as a blanket response for old API versions
- 415 Unsupported Media Type - If incorrect content type was provided as part of the request
- 422 Unprocessable Entity - Used for validation errors
- 429 Too Many Requests - When a request is rejected due to rate limiting

Facebook

```
HTTP Status Code: 200

{"type" : "OauthException", "message":"(#803) Some of the
aliases you requested do not exist: foo.bar"}
```

Twilio

```
HTTP Status Code: 401

{"status" : "401", "message":"Authenticate","code": 20003, "more
info": "http://www.twilio.com/docs/errors/20003"}
```

SimpleGeo

```
HTTP Status Code: 401

{"code" : 401, "message": "Authentication Required"}
```

Best Practices : Do's and Don'ts

5. Use Filtering, Sorting, and Pagination to Retrieve the Data Requested
   - Filtering: GET /users?age=30
   - Pagination: GET /users?page=3&results_per_page=20
   - Sorting: GET /users?sort_by=first_name&order=asc
   - Partial Data: GET /joe.smith/friends?fields=id,name,picture
6. Versioning a RESTful web API

```
Twilio              /2010-04-01/Accounts/
Salesforce          /services/data/v20.0/sobjects/Account
Facebook            ?v=1.0
```

   - How many versions and how long they should be maintained?

Use HATEOAS to enable navigation to related resources

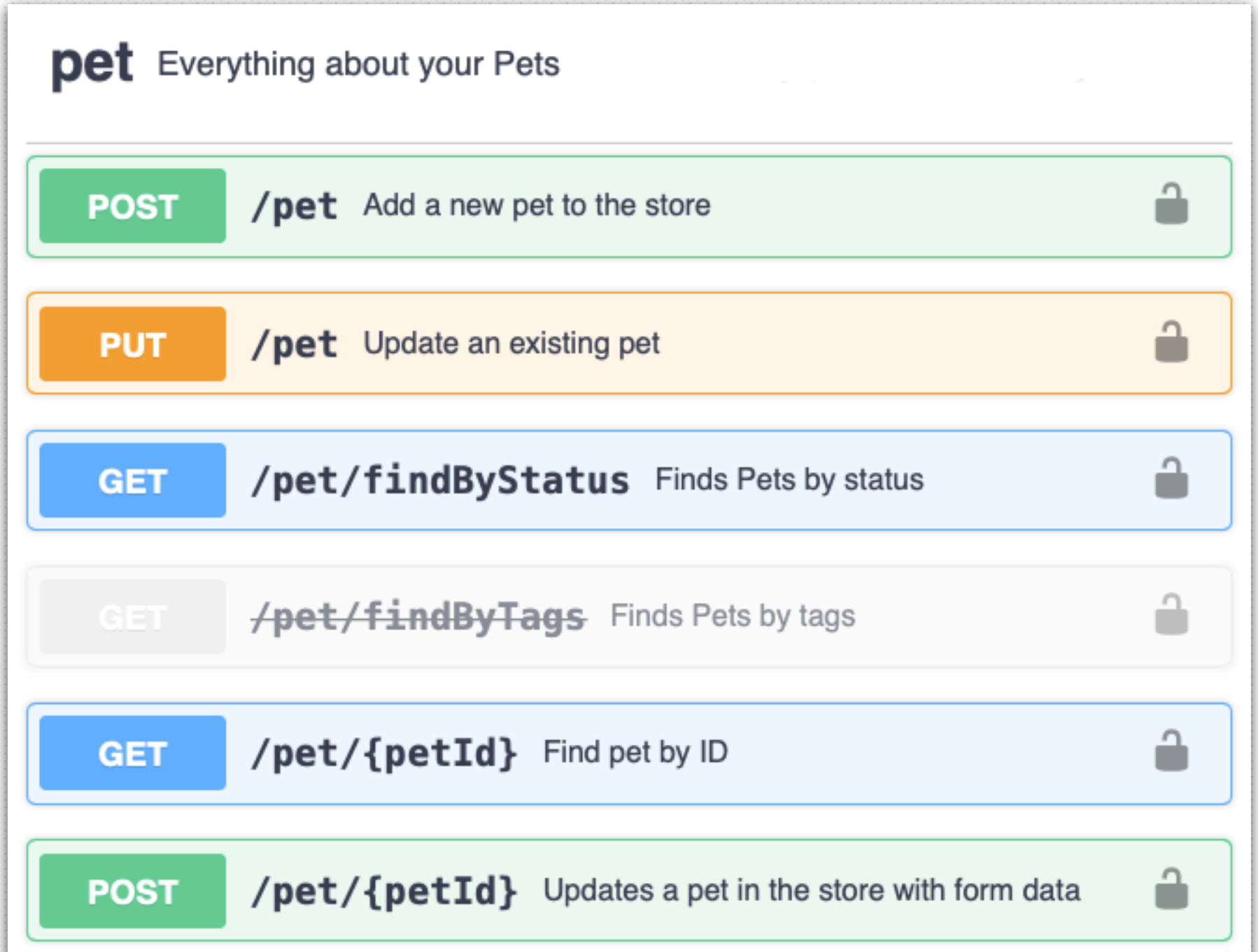
- Allows clients to navigate resources and discover available operations solely through the hyperlinks provided in responses

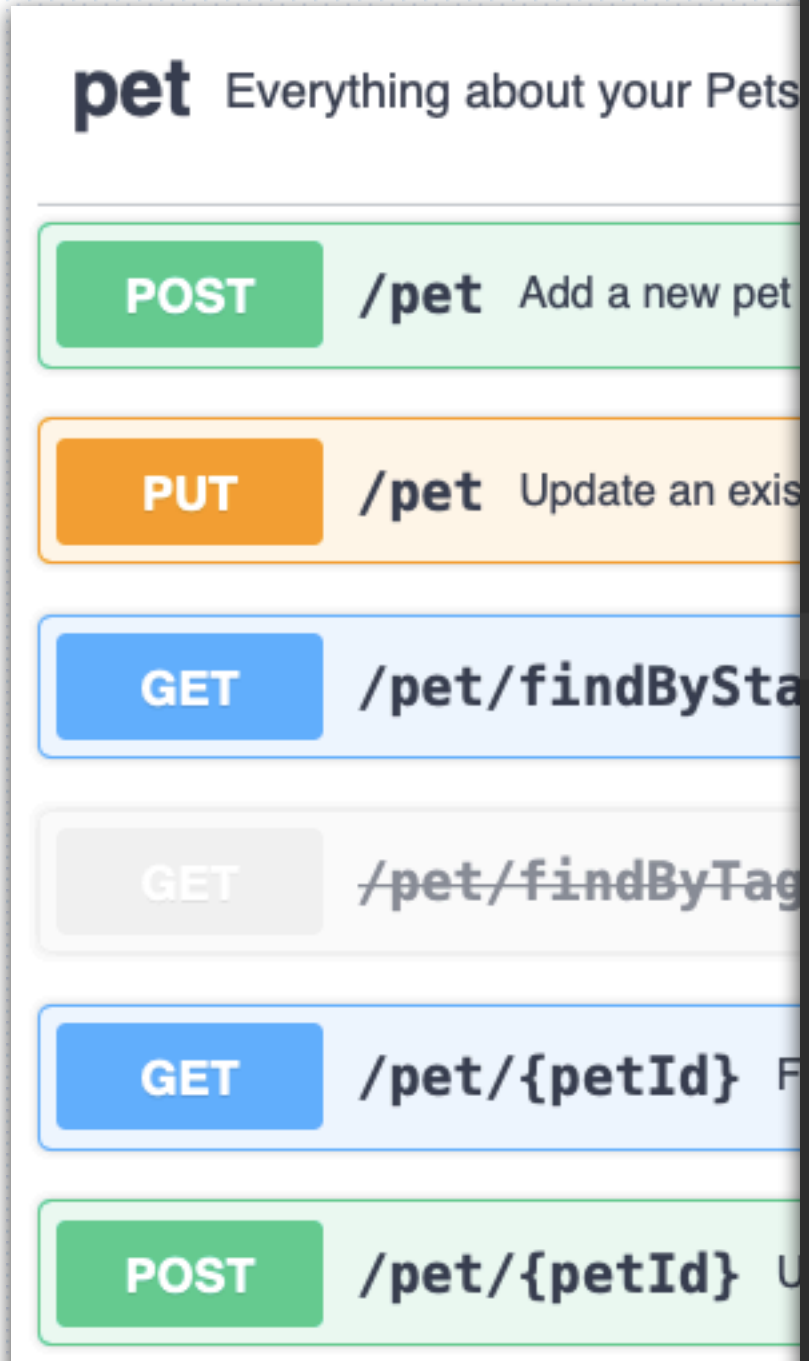
```
{
  "books": [
    {
      "id": 1,
      "title": "The Great Gatsby",
      "author": "F. Scott Fitzgerald",
      "links": {
        "self": "/books/1",
        "author": "/authors/1",
        "reviews": "/books/1/reviews"
      }
    },
    {
      "id": 2,
```

Accurate API Documentation

- Swagger is an open-source software framework backed by a large ecosystem of tools that helps developers design, build, document, and consume RESTful web services.
  - Provides a standard for service specification
  - Automatic documentation of Swagger helps a lot
- We will talk about REST (vs SOAP) services later!




pet Everything about your Pets
POST /pet Add a new pet to the store
PUT /pet Update an existing pet
GET /pet/findByStatus Finds Pets by status
GET /pet/findByTags Finds Pets by tags
GET /pet/{petId} Find pet by ID
POST /pet/{petId} Updates a pet in the store with form data




```yaml
- "http"
paths:
  /pet:
    post:
      tags:
      - "pet"
      summary: "Add a new pet to the store"
      description: ""
      operationId: "addPet"
      consumes:
      - "application/json"
      - "application/xml"
      produces:
      - "application/xml"
      - "application/json"
      parameters:
      - in: "body"
        name: "body"
        description: "Pet object that needs to be adde
          to the store"
        required: true
```

You Can Go For Code-First Approach

- Specs are designed at the first step in the former approach
- Some of developers like to write Java codes!
  - Code as documentation?
  - Mixing Swagger with Java

You Can Go For Code-First Approach

- Specs are d
- Some of de
  - Code as
  - Mixing S

```
@Api(value = "Swagger2DemoRestController", description = "REST Apis related to Student
@RestController
public class Swagger2DemoRestController {

    List<Student> students = new ArrayList<Student>();
    {
        students.add(new Student("Sajal", "IV", "India"));
        students.add(new Student("Lokesh", "V", "India"));
        students.add(new Student("Kajal", "III", "USA"));
        students.add(new Student("Sukesh", "VI", "USA"));
    }

    @ApiOperation(value = "Get list of Students in the System ", response = Iterable.cl
    @ApiResponses(value = {
            @ApiResponse(code = 200, message = "Suceess|OK"),
            @ApiResponse(code = 401, message = "not authorized!"),
            @ApiResponse(code = 403, message = "forbidden!!!"),
            @ApiResponse(code = 404, message = "not found!!!") })

    @RequestMapping(value = "/getStudents")
    public List<Student> getStudents() {
        return students;
    }
```

Don’t forget about API performance

- Effective caching
- Data compression methods
- Asynchronous processing
- Handling batch operations
- Measuring API performance & Analyzing API usage
- Monitoring API health and uptime
- Testing RESTful APIs (unit tests, integration tests, load tests, security tests) etc.